

Tut 02 – Memoria dinámica y punteros

Ingeniería en Inteligencia Artificial — Algoritmos y Estructura de Datos

2C 2026 · Docente: Ing. Magali Marijuan

- Tut 02 – Memoria dinámica y punteros
- Objetivos de la clase
- Los tres tipos de memoria
 - Memoria estática
 - Memoria automática (stack)
 - Memoria dinámica (heap)
- Punteros
 - Declaración y los operadores `*` y `&`
 - `nullptr`
 - Tamaño y tipo de los punteros
- Reservar memoria dinámica con `new`
- Arreglos dinámicos con `new[]`
- Aritmética de punteros
- Relación entre arreglos y punteros
- Liberar la memoria con `delete` y `delete[]`
- Referencias vs punteros
- `struct` / `class` en el heap: constructor y destructor
- Errores comunes
 - Memory leaks
 - Dangling pointers
- Valgrind
- Complejidad y ubicación en memoria

Objetivos de la clase

- Distinguir memoria estática, automática (stack) y dinámica (heap).
- Entender qué es un puntero y cómo se relaciona con las direcciones de memoria.
- Reservar y liberar memoria dinámica con `new` / `delete` y `new[]` / `delete[]`.
- Entender la relación entre arreglos y punteros, y la aritmética de punteros.
- Diferenciar punteros de referencias, y saber cuándo usar cada uno.
- Reconocer y evitar los errores más comunes: memory leaks y dangling pointers.
- Conocer Valgrind como herramienta para detectar problemas de memoria.

Los tres tipos de memoria

Memoria estática

Es la memoria reservada para variables **globales** y variables `static` , que existen durante **toda la ejecución del programa**.

```
int contadorGlobal = 0; // memoria estática

int main() {
    return 0;
}
```

Memoria automática (stack)

Es la memoria de las variables locales de una función, incluyendo los arreglos de tamaño fijo. Se reserva y libera **automáticamente** al entrar y salir de la función.

```
int main() {
    int x = 5;           // vive en el stack
    int arreglo[10];    // también vive en el stack
    return 0;
} // acá se libera automáticamente toda la memoria de x y arreglo
```

El tamaño de la memoria del stack se conoce en tiempo de compilación.

Memoria dinámica (heap)

Es memoria que el programa le pide **explícitamente** al sistema operativo en tiempo de ejecución, y que vive en una zona llamada **heap**.

Característica	Stack	Heap
¿Cuándo se reserva?	Automáticamente, al declarar la variable	Explícitamente, con <code>new</code>
¿Cuándo se libera?	Automáticamente, al salir de la función	Explícitamente, con <code>delete</code>

Característica	Stack	Heap
¿Se conoce el tamaño en compilación?	Sí	No necesariamente
Velocidad de acceso	Muy rápida	Más lenta
Riesgo principal	Desbordarse (<i>stack overflow</i>)	Fugas de memoria (<i>memory leaks</i>)

Pregunta para pensar: ¿por qué necesitaríamos memoria dinámica si ya tenemos el stack?
Pensá en un programa que lee un archivo y no sabe de antemano cuántos datos contiene.

Punteros

Un puntero es una variable que guarda una **dirección de memoria**, no un valor "normal" como un `int` o un `double`.

Declaración y los operadores `*` y `&`

```
int numero = 42;
int *p = &numero; // p guarda la direccion de memoria de numero
```

- El operador `&` (cuando se aplica a una variable) obtiene su **dirección de memoria**.
- El operador `*` (cuando se aplica a un puntero) accede al **valor apuntado** (operador de "desreferencia").

```
#include <iostream>

int main() {
    int numero = 42;
    int *p = &numero;

    std::cout << "Valor de numero: " << numero << std::endl;
    std::cout << "Direccion de numero: " << p << std::endl;
    std::cout << "Valor apuntado por p: " << *p << std::endl;

    *p = 100; // modifica numero a traves del puntero
    std::cout << "Nuevo valor de numero: " << numero << std::endl;

    return 0;
}
```

nullptr

Cuando un puntero todavía no apunta a nada válido, conviene inicializarlo con `nullptr` :

```
int *p = nullptr;

if (p == nullptr) {
    std::cout << "El puntero no apunta a nada todavia" << std::endl;
}
```

`nullptr` es la forma recomendada en C++ de representar un puntero que "no apunta a nada". Usalo siempre que declares un puntero que todavía no tiene un valor válido.

Tip: desreferenciar un puntero `nullptr` (hacer `*p` cuando `p == nullptr`) es un error grave en tiempo de ejecución (*segmentation fault*).

Tamaño y tipo de los punteros

El tamaño de un puntero no depende del tipo al que apunta, sino de la arquitectura del sistema (por ejemplo, 8 bytes en sistemas de 64 bits).

Los punteros sí tienen un **tipo**, que determina cómo se interpreta la memoria a la que apuntan y cómo funciona la aritmética de punteros:

```
int *pEntero;
double *pDoble;
char *pChar;
```

Pregunta para pensar: si `int` ocupa 4 bytes y `double` ocupa 8 bytes, ¿el puntero a `int` ocupa menos memoria que el puntero a `double` ?

Reservar memoria dinámica con `new`

Para pedir memoria dinámica en C++ usamos el operador `new` :

```
#include <iostream>

int main() {
    int *p = new int;    // reserva espacio para un int en el heap
    *p = 10;

    std::cout << *p << std::endl;

    delete p; // liberamos la memoria
    return 0;
}
```

También podemos inicializar el valor al reservarlo:

```
int *p = new int(10);
```

`new` no solo reserva la memoria: cuando reservamos un objeto (un `struct` o `class`), también llama automáticamente a su **constructor**. Vamos a ver esto más abajo.

Arreglos dinámicos con `new[]`

Para reservar un arreglo dinámico usamos `new[]` , indicando la cantidad de elementos:

```
#include <iostream>

int main() {
    int cantidad;
    std::cout << "Cuantos numeros va a guardar? ";
    std::cin >> cantidad;

    int *numeros = new int[cantidad];

    for (int i = 0; i < cantidad; i++) {
        numeros[i] = i * 2;
    }

    for (int i = 0; i < cantidad; i++) {
        std::cout << numeros[i] << std::endl;
    }

    delete[] numeros; // notar el [] al liberar
    return 0;
}
```

A diferencia de un arreglo estático (`int numeros[10]`), acá el tamaño (`cantidad`) puede conocerse recién en tiempo de ejecución.

Aritmética de punteros

Si `numeros` apunta al primer entero de un bloque de memoria:

```
*(numeros + 1)
```

accede al **segundo** entero. Aunque cada dirección de memoria representa 1 byte, sumar 1 a un puntero avanza `sizeof(tipo)` bytes, porque el compilador conoce el tipo del puntero.

```
int *p = new int[5] {10, 20, 30, 40, 50};

std::cout << *p << std::endl;           // 10
std::cout << *(p + 1) << std::endl;     // 20
std::cout << *(p + 2) << std::endl;     // 30

delete[] p;
```

Relación entre arreglos y punteros

Cuando declaramos un arreglo:

```
int miArreglo[10];
```

el nombre del arreglo se comporta como un puntero a su primer elemento. Por eso:

```
miArreglo[2]
```

es equivalente a:

```
*(miArreglo + 2)
```

El operador `[]` es, en el fondo, azúcar sintáctico (*syntactic sugar*) sobre la aritmética de punteros. Esto vale tanto para arreglos estáticos como para bloques reservados con `new[]`.

Liberar la memoria con `delete` y `delete[]`

Toda memoria reservada con `new` debe liberarse con `delete`, y toda memoria reservada con `new[]` debe liberarse con `delete[]`:

```
int *p = new int(5);
delete p;

int *arreglo = new int[10];
delete[] arreglo;
```

Importante: usar `delete` en algo reservado con `new[]` (o viceversa) es comportamiento indefinido. La regla es simple: si reservaste con `[]`, liberá con `[]`.

Reservo con	Libero con
<code>new Tipo</code>	<code>delete p;</code>
<code>new Tipo[n]</code>	<code>delete[] p;</code>

Pregunta para pensar: ¿qué diferencia hay entre la memoria del stack (que se libera sola) y la del heap (que hay que liberar a mano)? ¿Por qué el lenguaje no libera el heap automáticamente?

Referencias vs punteros

Ya vimos las referencias (&) en la clase anterior como forma de pasar variables a funciones. Una referencia es, conceptualmente, un **alias** de otra variable, mientras que un puntero es una variable que **guarda una dirección**.

	Puntero	Referencia
Puede ser <code>nullptr</code> / no inicializado	Sí	No, debe inicializarse siempre
Se puede "reasignar" a otra variable	Sí	No, queda ligada para siempre a la primera
Necesita <code>*</code> para acceder al valor	Sí	No, se usa como la variable original
Soporta aritmética (<code>p + 1</code>)	Sí	No

```
int a = 10;
```

```
int *p = &a;    // puntero: guarda la direccion de a
int &r = a;      // referencia: es "otro nombre" para a
```

```
*p = 20; // modifica a
r = 30;  // tambien modifica a
```

Tip: usen referencias cuando quieran que un parámetro sea "obligatorio" y siempre válido (por ejemplo, en funciones). Usen punteros cuando necesiten representar "puede no haber nada" (`nullptr`), reasignar a qué apuntan, o hacer aritmética.

struct / class en el heap: constructor y destructor

Los `struct` y `class` también se pueden reservar dinámicamente con `new`. Cuando eso pasa, C++ llama automáticamente al **constructor**, y al hacer `delete` llama al **destructor**.


```

#include <iostream>

struct Punto {
    int x;
    int y;

    Punto(int xInicial, int yInicial) { // constructor
        x = xInicial;
        y = yInicial;
        std::cout << "Se creo un Punto" << std::endl;
    }

    ~Punto() { // destructor
        std::cout << "Se destruyo un Punto" << std::endl;
    }
};

int main() {
    Punto *p = new Punto(3, 4);

    std::cout << "(" << p->x << ", " << p->y << ")" << std::endl;

    delete p; // acá se llama al destructor
    return 0;
}

```

- `p->x` es equivalente a `(*p).x` : accede al campo `x` del struct apuntado por `p`.
- El **constructor** se ejecuta automáticamente al crear el objeto (con `new` o en el stack).
- El **destructor** se ejecuta automáticamente al destruirlo (con `delete`, o al salir del scope si vive en el stack).

Esta idea, de que un objeto se encarga de gestionar sus propios recursos (por ejemplo, reservando memoria en el constructor y liberándola en el destructor) se llama **RAII** (*Resource Acquisition Is Initialization*). Es uno de los pilares de C++ moderno y lo van a ver en profundidad más adelante en la materia.

Errores comunes

Memory leaks

Un *memory leak* (fuga de memoria) ocurre cuando reservamos memoria dinámica y **nunca la liberamos**.

```
void funcionConLeak() {  
    int *p = new int(5);  
    // nos olvidamos del delete p;  
} // p (el puntero) se destruye, pero la memoria que apuntaba queda reservada para siempre
```

Cada `new` sin su `delete` correspondiente es un memory leak. En programas que corren mucho tiempo (o que reservan memoria repetidamente), esto puede agotar toda la memoria disponible.

Dangling pointers

Un *dangling pointer* (puntero colgante) es un puntero que sigue apuntando a una dirección de memoria que **ya fue liberada**.

```
int *p = new int(10);  
delete p;  
  
std::cout << *p << std::endl; // comportamiento indefinido: p es un dangling pointer
```

Tip: después de hacer `delete p;`, es una buena práctica asignar `p = nullptr;` para evitar usarlo por error. Acceder a un puntero `nullptr` sigue siendo un error, pero uno mucho más fácil de detectar que un dangling pointer.

Pregunta para pensar: ¿qué diferencia hay entre desreferenciar un puntero `nullptr` y desreferenciar un dangling pointer? ¿Cuál te parece más difícil de debuggear?

Valgrind

Valgrind es una herramienta que permite analizar el uso de memoria de un programa y detectar, entre otras cosas:

- Accesos a memoria inválida.

- Memoria reservada con `new` / `new[]` que nunca se liberó (memory leaks).
- Usos de memoria ya liberada (dangling pointers).

Para correrlo sobre un ejecutable:

```
valgrind -s --error-exitcode=1 --leak-check=full --show-leak-kinds=all --track-origins:
```

Usar Valgrind sobre cada entrega es una buena costumbre: muchos bugs de memoria "no se ven" simplemente corriendo el programa, pero Valgrind los detecta igual.

Complejidad y ubicación en memoria

Para cerrar, un resumen de dónde vive cada cosa y qué tan "cara" es:

Operación	Ubicación	Complejidad temporal
Declarar variable local (<code>int x</code>)	Stack	$O(1)$
Declarar arreglo estático (<code>int a[n]</code>)	Stack	$O(1)$ para reservar (el tamaño ya se conoce en compilación)
<code>new Tipo</code> / <code>new Tipo[n]</code>	Heap	$O(1)$ amortizado para reservar; $O(n)$ si el constructor inicializa cada elemento
Acceder a <code>arreglo[i]</code>	Stack o Heap	$O(1)$
<code>delete</code> / <code>delete[]</code>	Heap	$O(1)$ o $O(n)$ si hay que destruir n objetos

La ventaja del heap no es la velocidad (acceder al stack suele ser más rápido), sino la **flexibilidad**: nos permite manejar tamaños que solo se conocen en tiempo de ejecución, y hacer que los datos "sobrevivan" más allá del scope de una función.